Análisis teórico: pertinencia y límites de Q-learning y DQN

Componente obligatorio sin implementación. Se consideran las configuraciones por defecto de Gymnasium 1.4.0: BipedalWalker-v3 y MuJoCo v5. Las dimensiones se contrastaron con la documentación oficial y los archivos de la versión instalada; no se entrenaron estos entornos. Los tamaños de tablas y rejillas siguientes son cálculos propios, no resultados experimentales.

1. El algoritmo debe corresponder al espacio de acciones

Q-learning tabular guarda un valor por cada par estado-acción. DQN aproxima esos valores mediante una red, pero su versión estándar entrega un valor por acción discreta y calcula un máximo sobre todas ellas. En CartPole hay dos acciones: la tabla con resolución 2/2/6/6 contiene 144 estados y 288 valores Q. En LunarLander discreto hay ocho componentes observados y cuatro acciones: DQN puede compartir información entre observaciones próximas sin enumerarlas. Dos componentes de LunarLander son indicadores de contacto, por lo que describir las ocho observaciones como ocho variables estrictamente continuas sería impreciso.

En locomoción, las acciones son vectores continuos: especifican simultáneamente controles de varias articulaciones. No hay una lista finita que el DQN estándar pueda enumerar directamente. Si se asignan k niveles a cada una de d componentes, aparecen k^d acciones conjuntas, no k·d. [G1-G4]

Entorno	Dimensión de observación	Controles d	k=3: acciones	k=5: acciones	k=11: acciones
BipedalWalker-v3	24	4	81	625	14641
Hopper-v5	11	3	27	125	1331
HalfCheetah-v5	17	6	729	15625	1771561
Ant-v5	105	8	6561	390625	214358881

Ant-v5 incluye por defecto 78 componentes de fuerzas de contacto, además de 27 posiciones y velocidades; otros argumentos o versiones cambian esa dimensión. BipedalWalker mezcla magnitudes físicas, contactos y lidar. No todas las observaciones son variables continuas independientes. [G1,G4]

La rejilla de acciones es posible en principio, especialmente para Hopper, pero restringe el control y encarece la búsqueda y la capa de salida de DQN. En Ant, once niveles exigirían más de 214 millones de valores de salida por observación. Una rejilla más gruesa abarata el cálculo a costa de perder precisión; aumentar la resolución produce crecimiento exponencial. No se afirma que discretizar sea matemáticamente imposible, sino poco adecuado para el presupuesto de este proyecto.

2. Por qué tampoco escala una tabla de estados

Discretizar m observaciones en b intervalos produce, en una rejilla completa, b^m estados y b^m·k^d valores Q. Para Hopper, diez intervalos por observación y cinco niveles por control dan 10^11·125 entradas: con float32 ocupan 5·10^13 bytes, aproximadamente 50 TB decimales, sin contar estructuras auxiliares. Visitar suficientemente esos pares es aún más costoso. Las correlaciones físicas hacen que parte de la rejilla sea inalcanzable; no eliminan el problema de una tabla densa.

Agrupar estados reduce memoria, pero puede unir situaciones que necesitan acciones distintas: dos velocidades dentro del mismo intervalo pueden requerir correcciones diferentes. Es el compromiso entre resolución y cobertura. Nuestros resultados de CartPole son compatibles con este mecanismo: aumentar intervalos no mejoró automáticamente el retorno bajo 200000 pasos. La observación experimental no identifica una causa única ni demuestra que una granularidad sea universalmente óptima.

3. Familias apropiadas para control continuo

- **PPO:** aprende una política parametrizada que puede generar vectores de acciones continuas. Su objetivo recortado limita el incentivo a cambios excesivos de política en los datos usados; no garantiza por sí solo estabilidad. Es una alternativa sencilla de actor-crítico, aunque requiere recoger nuevos datos en sus rondas de optimización. [P1]

- **SAC:** actor-crítico fuera de política, reutiliza experiencia y combina recompensa con entropía. Una política estocástica continua evita enumerar una rejilla y favorece exploración. Es una opción pertinente cuando importa aprovechar interacciones costosas, sin garantizar que siempre supere a PPO. [P2]

- **TD3:** actor continuo determinista y dos críticos; utiliza el menor valor objetivo, actualizaciones retrasadas de la política y suavizado de acciones objetivo para controlar errores de aproximación. También reutiliza experiencia. Comparte la motivación de limitar sobreestimación con Double DQN, pero no es el mismo algoritmo ni convierte Double DQN en un controlador continuo. [P3]

Estas familias resuelven la representación de acciones continuas mediante un actor, no mediante una salida por cada combinación de controles. La elección práctica dependería de coste de interacción, exploración y ajuste; aquí se justifica su pertinencia, sin afirmar rendimientos medidos en nuestros equipos.

4. Por qué una tabla tampoco sirve para píxeles de Atari

Considérese una entrada habitual preprocesada de cuatro imágenes de 84×84 en escala de grises: son 28224 componentes con 256 intensidades posibles. El conjunto de todas las combinaciones tiene 256^28224, aproximadamente 10^67970 entradas visuales posibles, antes de multiplicar por acciones. Es una cota combinatoria, no el número de imágenes físicamente alcanzables en un juego. Incluso a un bit por píxel quedarían 2^28224 combinaciones. Reducir colores no vuelve viable una tabla exhaustiva. [G5,G6]

Una tabla tampoco comparte automáticamente lo aprendido entre imágenes parecidas. DQN con una red convolucional comparte rasgos espaciales y aproxima Q sin almacenar un valor distinto para cada imagen. Apilar imágenes aporta información sobre movimiento, aunque no garantiza observabilidad completa. Atari conserva acciones discretas: por eso DQN es pertinente con píxeles, mientras que su dificultad en locomoción incluye además el espacio continuo de acciones. Nuestro MLP de CartPole/LunarLander no constituye por sí solo una implementación DQN para Atari. [P4]

Referencias verificadas

- [G1] Gymnasium: BipedalWalker.
- [G2] Gymnasium: Hopper.
- [G3] Gymnasium: HalfCheetah.
- [G4] Gymnasium: Ant.
- [G5] Gymnasium: AtariPreprocessing.
- [G6] Gymnasium: FrameStackObservation.
- [P1] Schulman et al. (2017). Proximal Policy Optimization Algorithms.
- [P2] Haarnoja et al. (2018). Soft Actor-Critic.
- [P3] Fujimoto et al. (2018). Addressing Function Approximation Error in Actor-Critic Methods.
- [P4] Mnih et al. (2013). Playing Atari with Deep Reinforcement Learning.

Esta sección satisface el análisis sin implementación; se integrará después en el artículo final de 6-8 páginas. La discusión experimental y los intervalos se presentan por separado para no confundir cálculos de escalabilidad con mediciones de agentes entrenados.